INF3132 – Programmation Orientée Objet
S5 2023–2024

esiea

Cours 3:

Polymorphisme
Classes abstraites et Interfaces
Exceptions

Felicia IONASCU
Siba HAIDAR
Aylen RICCA

Plan

1. Rappels
2. Polymorphisme
3. Classes abstraites et Interfaces
4. Exceptions

Plan

1. Rappels
2. Polymorphisme
3. Classes abstraites et Interfaces
4. Exceptions

Rappels : les constructeurs

- Avant d'être utilisé, chaque objet doit être initialisé en appelant l'un de ses Constructeurs

```
class Student {
  Student(int i) {           //constructor
     System.out.println("Creating Student number " + i);
  }
}
public class SimpleConstructorDemo {
    public static void main(String[] args) {
        for(int i = 0; i < 10; i++)
           Student st = new Student(i);     //object creation and memory allocation
    }
}
```

- Ce code crée 10 référents «`st`» (sur la pile) et alloue 10 objets de la classe `Student` (sur le tas)

Rappels : la mémoire

- Les types primitifs et les référents :
  - sont alloués sur la Pile (automatiquement)
  - leur durée de vie est celle du bloc { }
  - mémoire libérée automatiquement
- Les objets :
  - sont alloués sur le Tas (manuellement, avec `new`)
  - la mémoire est libérée par le GC
- Les attributs, les référents et les méthodes statiques :
  - sont alloués dans une zone de mémoire dédiée dans la JVM
  - leur durée de vie est égale à celle du programme
- Les méthodes des classes :
  - sont allouées dans une zone de mémoire dédiée dans la JVM
  - leur durée de vie est égale à celle du programme

Rappels : le Ramasse-Miettes

- Le Ramasse-Miettes libère la mémoire allouée sur le Tas (les objets) quand :
  - On s'approche de la limite supérieure de la mémoire dont la JVM dispose
  - Les objets n'ont plus de référent
  - Sur appel de la méthode `System.gc()`
- Le Ramasse-Miettes ne s'occupe pas de la finalisation correcte des objets (e.g., ce n'est pas lui qui va clore les fichiers ouverts)
- On ne peut pas se fier au fait que la méthode `finalize()` sera exécutée, car on ne contrôle pas le passage du Ramasse-Miettes

Rappels : héritage

- L'héritage consiste à définir les classes comme un empilement de concepts avec des caractéristiques communes, du plus abstrait au plus spécifique
- Une classe fille peut hériter d'une seule classe mère
- Une classe fille peut avoir à son tour plusieurs classes filles

```
class Vehicle {
     protected int position;
     void go() { position = 1; }
}
class Plane extends Vehicle {
     void  land() { position = 0; }
}
class Car extends Vehicle {
     void stop() { position = 0; }
}
```

Rappels : encapsulation

- On regroupe les classes en package (*library units*) avec le mot-clef `package` en début du fichier
  - `package com.mycompany.myframework.complex`
- On protège l'accès aux attributs et méthodes d'une classe par des modificateurs d'accès

Modificateur d'accès	Visibilité des éléments
`Public`	Classes à l'intérieur et à l'extérieur du package
`(friendly)`	Classes à l'intérieur du package
`Protected`	Classe courante et ses classes filles, à l'intérieur et à l'extérieur du package
`Private`	Classe courante

Quiz – héritage et encapsulation



Cours 3 - Question 1

- QCM - Rappels (5 min)

Plan

1. Rappels
2. Polymorphisme
3. Classes abstraites et Interfaces
4. Exceptions

Rappel : les 4 concepts de base de la POO

1. **Abstraction** – mécanisme qui permet de créer un type générique regroupant les caractéristiques communes des entités => les classes

2. **Héritage** – une nouvelle classe peut inclure les caractéristiques d'une autre classe

3. **Encapsulation** – définit les méthodes d'accès aux données et permet de cacher l'implémentation

4. **Polymorphisme** – fournir une interface unique à des entités pouvant avoir différents types

Polymorphisme

- Du grec ancien :
  - πολύμορφος / *polúmorphos* = plusieurs formes

- Concepts :
  - Polymorphisme par héritage
  - Redéfinition de méthode (*method overriding* en Anglais)

La *redéfinition de méthode*

- *Overriding* =
  redéfinition de méthode

```
class Vehicle {
    protected int position;
    void go() { position = 1; }
}
class Plane extends Vehicle {
   void  go() { position = 3; }
}
class Car extends Vehicle {
   void go() { position = 2; }
}
```

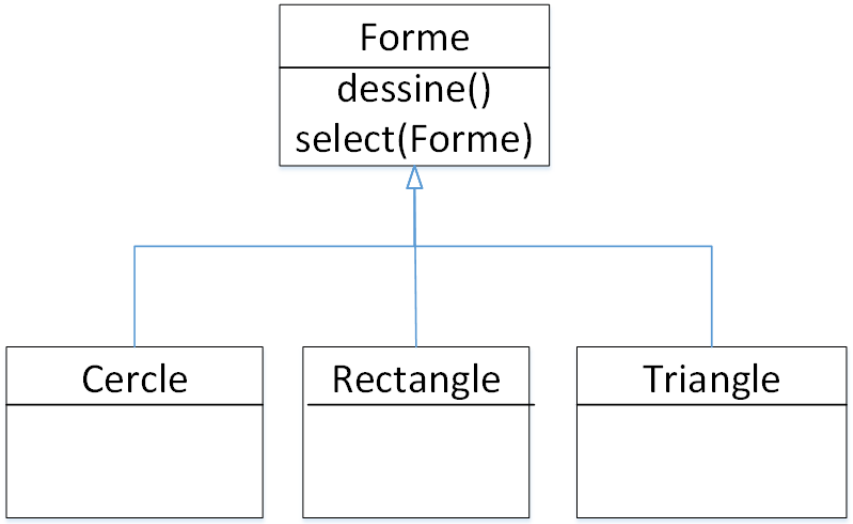
- *Overloading = surcharge de méthode*
  - Une méthode surchargée doit avoir des arguments différents en nombre et en type

```
class Vehicle {
    protected int position;
    void go() { position = 1; }
}
class Plane extends Vehicle {
   void  go(int nextPosition) {
       position = nextPosition;
   }
}
class Car extends Vehicle {
   int timeout;
   void go(int nextPosition, int when) {
       position = nextPosition;
        timeout = when;
   }
}
```

Polymorphisme par héritage (1/5)

Problème

- On veut écrire un programme qui :
  - Dessine toutes les formes géométriques représentées dans ce diagramme:



  - Est facilement extensible (on peut rajouter une nouvelle forme avec un minimum d'impact)

Polymorphisme par héritage (2/5)

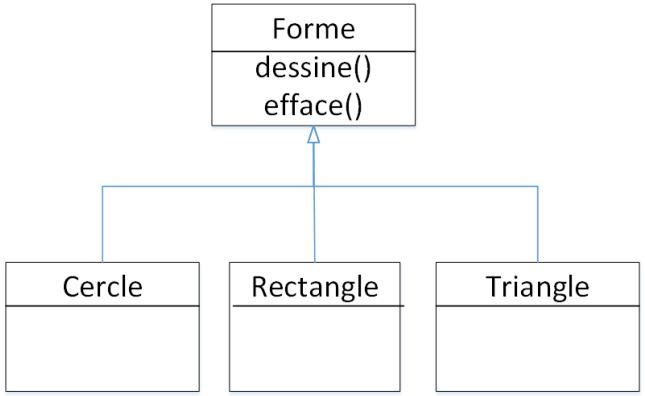
Approche naïve

```
class Forme {
  public void dessine() { System.out.println("Forme.dessine"); }
}
class Cercle extends Forme {
   public void dessine() { System.out.println("Cercle.dessine"); }
}
class Triangle extends Forme {
   public void dessine() { System.out.println("Triangle.dessine"); }
}
public class Maths {
  public static void geometrie(Cercle c) { c.dessine(); }        //dessine un cercle
  public static void geometrie(Triangle t) { t.dessine(); }      //dessine un triangle
  public static void main(String[] args) {
        Cercle monCercle = new Cercle();
        Triangle monTriangle = new Triangle();
        geometrie(monCercle);
        geometrie(monTriangle);
  }
}
```

Polymorphisme par héritage (3/5)

Héritage pur versus Extension

Héritage pur

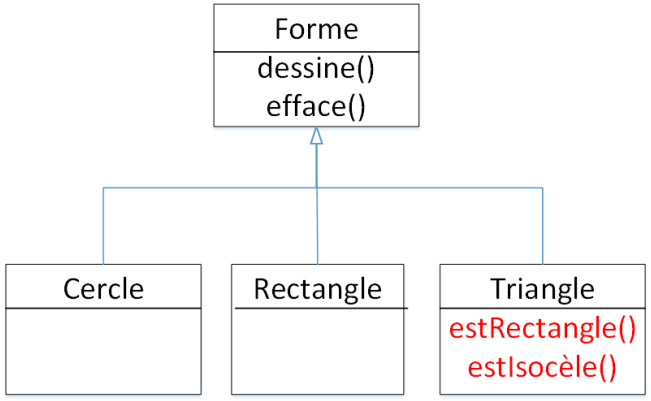


classe qui utilise Forme → message → Cercle, Rectangle, Triangle ou un nouveau type de Forme

Relation « est un »
ou Substitution pure

Le Polymorphisme est possible

Extension



classe qui utilise Forme → message → dessine() efface() ~~estRectangle() estIsocèle()~~

Relation « est comme un »

Polymorphisme par héritage (4/5)

Upcasting

```
class Forme {
  public void dessine() {System.out.println("Forme.dessine");}
}
class Cercle extends Forme {
  public void dessine() {System.out.println("Cercle.dessine");}
}
class Maths {
  static void geometrie(Forme i) {      //l'argument de geometrie est Forme
     i.dessine();
  }
  public static void main(String[] args) {
     Cercle monCercle = new Cercle();
     geometrie(monCercle);      //mais on appelle la méthode geometrie avec Cercle : Upcasting
  }
}
```

- Le *upcasting* est **la conversion du référent de la classe fille vers le référent de sa classe mère**
- Le *upcasting* est **toujours sûr** car la classe fille a au moins les méthodes de la classe mère
  - un Cercle est une Forme
  - => tous les messages qu'on peut envoyer à la classe mère peuvent être envoyés à la classe fille
- Le contraire (*downcasting*) devrait être possible avec un bon design (principe de *Liskov*)

Polymorphisme par héritage (5/5)

Early binding et *Late binding*

- Le *binding* est la connexion qui est faite entre l'appel d'une méthode et sa déclaration
- *Early binding* = la connexion est faite à la compilation, les appels de méthode sont plus efficaces
- *Late binding* = la connexion est faite à l'exécution, la flexibilité est plus grande

- *En C : Early binding*
- En Java : toujours *Late binding*, sauf pour les méthodes `private`, `static`, `final`

- Grâce à la *Late binding*:
  - on peut écrire du code qui appelle la classe mère et toutes les classes fille vont fonctionner de la même manière (simplicité)
  - on peut rajouter autant de classes fille qu'on veut sans modifier le code qui appelle la classe mère (extensibilité)

Polymorphisme : exemple 1

```
public class Vehicle {
   protected int position;
   public void go() {  position = 0; }
}
public class Plane extends Vehicle {
      public void go() { position = 1; } //redéfinition
}
public class Airbus extends Plane {
    public void go() { position = 2; } //redéfinition
}
public class AirbusLauncher {
      public static void main (String args[])     {
            Vehicle ab = new Airbus();
            ab.go();                  //résultat: position = 2
      }
}
```

- La JVM appelle la méthode correcte pour l'objet crée (Airbus)
- Elle n'appelle pas la méthode qui est définie par le type du référent (Vehicle)

=> appel de méthode virtuelle

Polymorphisme : exemple 2

```
class Forme {
        void dessine() {}
}
class Cercle extends Forme {
        void dessine() { System.out.println("Cercle.dessine()"); }
}
class Rectangle extends Forme {
        void dessine() { System.out.println("Rectangle.dessine()"); }
}
public class Formes {
        public static Forme genAleatoire () {
                switch((int)(Math.random() * 2)) {
                        default:
                        case 0: return new Cercle();
                        case 1: return new Rectangle();
                }
        }
        public static void main(String[] args) {
                Forme[] s = new Forme[9];
                for(int i = 0; i < s.length; i++) s[i] = genAleatoire();
                for(int i = 0; i < s.length; i++) s[i].dessine();
        }
}
```

- La JVM appelle la méthode correcte pour l'objet crée (`Cercle` ou `Rectangle`)
- Elle n'appelle pas la méthode qui est définie par le type du référent (`Forme`).

=> appel de méthode virtuelle

Polymorphisme : exemple 3

```
public class Bicycle {
    private int cadence;
    private int gear;
    private int speed;

    public Bicycle(int startCadence, int startSpeed, int startGear){
        gear = startGear;
        cadence = startCadence;
        speed = startSpeed;
    }

    public void printDescription(){
        System.out.println("\nBike is " + "in gear " + this.gear
            + " with a cadence of " + this.cadence +
            " and travelling at a speed of " + this.speed + ". ");
    }
}
```

```
public class RoadBike extends Bicycle{
   private int tireWidth;  // In millimeters (mm)

    public RoadBike(int startCadence, int startSpeed,
                    int startGear, int newTireWidth){
        super(startCadence, startSpeed, startGear);
        this.tireWidth = newTireWidth;
    }

    public void printDescription(){
        super.printDescription();
        System.out.println("The RoadBike" + " has " + getTireWidth() +
            " MM tires.");
    }
}
```

```
public class MountainBike extends Bicycle {
    private String suspension;

    public MountainBike(int startCadence, int startSpeed,
                int startGear, String suspensionType){
        super(startCadence, startSpeed, startGear);
        this.suspension = suspensionType;
    }
    public void printDescription() {
        super.printDescription();
        System.out.println("The " + "MountainBike has a " +
            getSuspension() + " suspension.");
    }
}
```

```
public class TestBikes {
  public static void main(String[] args){
    Bicycle bike01, bike02, bike03;

    bike01 = new Bicycle(20, 10, 1);
    bike02 = new MountainBike(20, 10, 5, "Dual");
    bike03 = new RoadBike(40, 20, 8, 23);
    bike01.printDescription();
    bike02.printDescription();
    bike03.printDescription();
  }
}
```

```
D:\>java TestBikes

Bike is in gear 1 with a cadence of 20 and travelling at a speed of 10.

Bike is in gear 5 with a cadence of 20 and travelling at a speed of 10.
The MountainBike has a Dual suspension.

Bike is in gear 8 with a cadence of 40 and travelling at a speed of 20.
The RoadBike has 23 MM tires.
```

Exercice Polymorphisme



Cours 3 - Question 3

- Polymorphism

Plan

1. Rappels
2. Polymorphisme
3. Classes abstraites et Interfaces
4. Exceptions

Classes Abstraites (1/2)

- Dans l'exemple de polymorphisme, l'implémentation de la méthode `dessine()` de la classe `Forme` n'est jamais utilisée
- Le rôle de la classe `Forme` est de créer une interface commune pour toutes les classes dérivées d'elle

La classe `Forme` est une **classe abstraite**

- **On crée une classe abstraite quand on veut manipuler un set de classes à travers leur interface commune, tout en empêchant la création d'un objet de cette classe**
- Toutes les méthodes des classes dérivées qui ont la signature de la déclaration dans la classe mère seront appelées avec le mécanisme *dynamic binding (late binding)*
- En Java, on déclare une classe abstraite avec le qualificatif `abstract`

Classes Abstraites (2/2)

- Le compilateur garantit qu'on ne peut pas instancier des objets d'une classe abstraite (erreur de compilation)
- Si la classe fille n'implémente pas les méthodes abstraites de la classe mère, elle est aussi abstraite

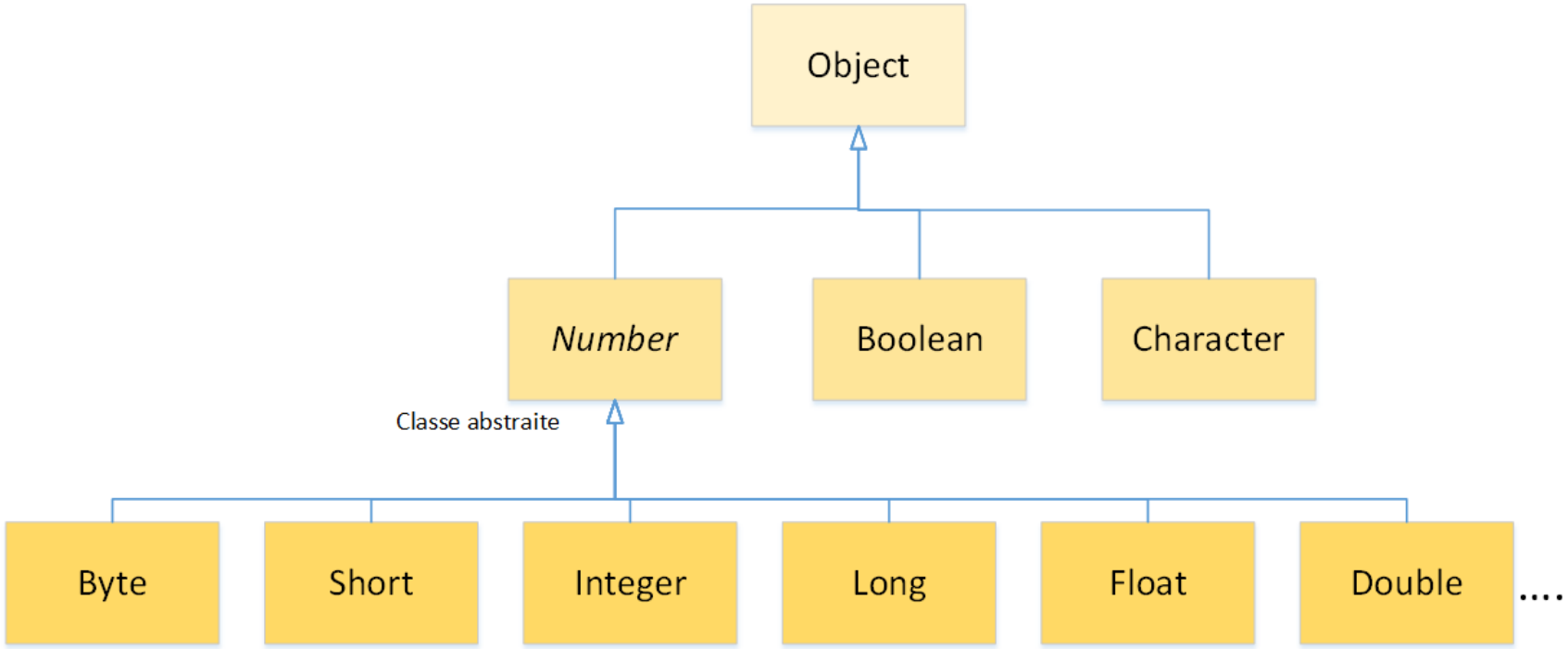
```
abstract class Forme {
  public abstract void dessine();       //la méthode abstraite n'a pas d'implémentation
}

class Cercle extends Forme {
   public void dessine() { System.out.println("Cercle.dessine"); }
}

public class Maths {
  public static void geometrie(Forme f) { f.dessine(); }  //dessine une forme
  public static void main(String[] args) {
        Forme monCercle = new Cercle();
        geometrie(monCercle);                              //Upcasting
  }
}
```

Exemple de classe abstraite dans la JDK

- Les classes Wrapper



- Qu'est-ce qui nous indique le fait que `Number` est *`abstract`* ?

Exercice



Cours 3 - Question 4

- Classe abstraite

Interface (1/2)

- Une interface est un **type abstrait**
- Une interface dit : « Voilà quelles sont les méthodes des classes qui implémentent cette interface » (un contrat)
- Elle établit un **protocole de communication** ou **décrit un comportement**

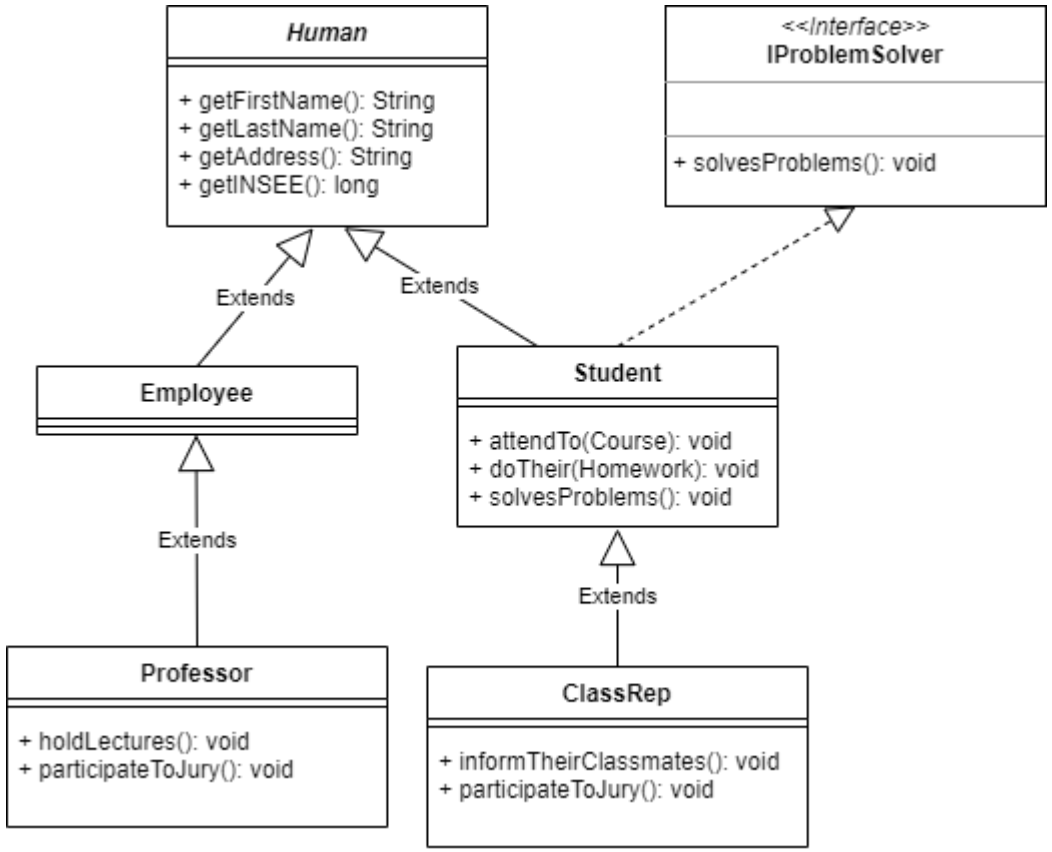
- On déclare une interface avec le mot clef `interface` :

```
interface MyInterface
```

- On déclare qu'une classe implémente une interface avec le mot clef `implements` :

```
class MyClass implements MyInterface
```

Interface : exemple




```
abstract class Human
interface IProblemSolver
class Employee extends Human
class Student extends Human implements IProblemSolver
class Professor extends Employee
class ClassRep extends Student
```

Interface (2/2)

- Une interface peut contenir :
  - Des méthodes abstraites
  - Des méthodes par défaut
    - elles ont une implémentation dans l'interface
    - Permettent de rajouter une nouvelle méthode dans une interface qui est automatiquement disponible dans toutes les implémentations
  - Des méthodes statiques
  - Des déclarations de constantes (`public static final` de manière implicite)

```
public interface Printable {
    String TYPE = "Paper";
                // le compilateur transforme en : public static final String TYPE = "Paper";
    void print();
                // le compilateur transforme en : public abstract void print();
    default void messageToUser()  {
        System.out.println("Printing on paper");    //implémentation fournie
    }
}
```

L'interface Comparable (1/4)

https://docs.oracle.com/en/java/javase/16/docs/api/java.base/java/lang/Comparable.html

- Les objets d'une classe qui réalise cette interface obéissent à une relation d'ordre (ordre naturel)
- Contient une seule méthode :

```
int compareTo(T obj)
```

Compare l'objet courant (`this`) avec l'objet passé en paramètre

- La méthode doit retourner le signe de la soustraction virtuelle `this - obj` :
  - `-1` si `this < obj`, `0` si `this == obj`, `+1` si `this > obj`
- La méthode doit avoir les propriétés suivantes, pour toutes instances de la classe nommées `o1`, `o2` et `o3` :
  - `o1.compareTo(o1) == 0;`
  - `o1.compareTo(o2) == - o2.compareTo(o1);`
  - `o1.compareTo(o2) > 0` et `o2.compareTo(o3) > 0` implique `o1.compareTo(o3) > 0`

L'interface Comparable (2/4)

```
class Kid implements Comparable {
private int age;
public Kid(int age) {
       if (age <= 0)
            throw new Exception();
       this.age = age;
}
public int compareTo(Object o) {
    Kid k = (Kid)o;
    if (this.age == k.age)
        return 0;
    else if (this.age > k.age)
        return 1;
    else
        return -1;
    }
}
```

- Quel est le critère de comparaison?
  - C'est la classe qui le connait et elle l'utilise dans la méthode compareTo(...)

```
public static void main(String[] args) {
        Kid kid1 = new Kid(10);
        Kid kid2 = new Kid(20);

        System.out.println("kid1, kid2: «
                + kid1.compareTo(kid2));
 }
```

```
kid1, kid2: -1
```

L'interface Comparable (3/4)

```
public class ComparableIntegers {

    public static void main(String[] args) {

        Integer valA = Integer.valueOf(30);
        Integer valB = Integer.valueOf(32);

        int comparisonA = valA.compareTo(valB);
        int comparisonB = valB.compareTo(valA);

        System.out.println(comparisonA);
        System.out.println(comparisonB);
    }
}
```



Cours 3 - Question 5

L'interface Comparable (4/4)

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	|
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

`"abc" compareTo "ABC"`	`32`
`"ABC" compareTo "abc"`	`-32`

```
String str = "abc"
```

est équivalent à :

```
char data[] = {'a', 'b', 'c'};
String str = new String(data);
```

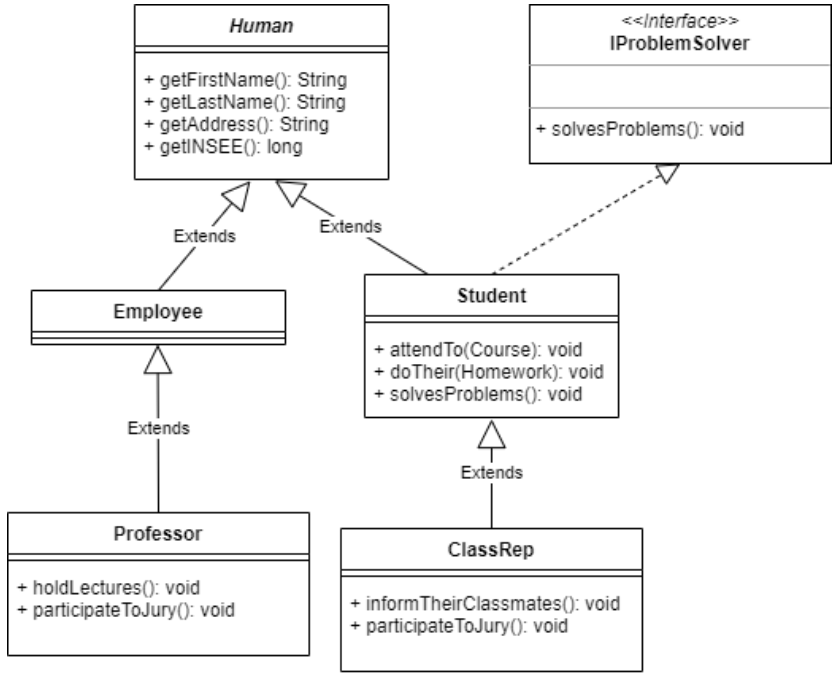
- Comparaison lexicographique, binaire
- résultat positif => dans la table des valeurs ASCII l'objet courant (this) se trouve "après" l'objet passé en paramètre

- Pour comparer des Strings contenant des caractères Unicode, on ne peut pas utiliser `compareTo` car les valeurs Unicode ne correspondent pas à l'ordre lexicographique => on peut utiliser la classe `Collator` pour trier des Strings
  - https://docs.oracle.com/javase/tutorial/i18n/text/collationintro.html

Classe abstraite ou interface?

- Si nous avons des **concepts en relation ou du code à mettre en commun entre plusieurs classes => classe abstraite**
- Si nous avons un comportement commun à des classes **sans rapport entre elles => interface**
- Si nous avons besoin d'un **contrat entre classes => interface**
- (En Java) Si nous avons besoin d'**héritage multiple => interface**

- Les interfaces peuvent jouer le rôle d'**API** (Application Programming Interface)
  - Les interfaces réalisent un **couplage faible** entre les entités



Classe abstraite ou interface ?

- Allez sur wooclap.com
  - Question 1, 2 et 3



Utiliser une interface en tant que type

- Une interface est un nouveau type
- On peut utiliser un nom d'interface en tant que référent
- Si un référent est de type interface, les objets auxquels il fait référence doivent être des instances de classes qui implémentent l'interface

```
public interface Printable {
    String TYPE = "Paper";
    void print();
    default void messageToUser()  {
        System.out.println ("Printing on paper");
    }
}
```

```
public class Printer {
  private Printable currentSheet;
  void printCurrentSheet() {
        currentSheet.print();
  }
  void setCurrentSheet(Printable p) {
        currentSheet = p;
  }
}
```

```
public class Paper implements Printable {
    public void print() {
        System.out.println("Print on paper");
    }
}
```

```
public class PrinterTest {
    public static void main(String[] args) {
        Printer printer = new Printer();
        Printable p = new Paper();
        printer.setCurrentSheet(p);
        printer.printCurrentSheet();
    }
}
```

Interfaces qui évoluent (1/2)

- Problème :
  - Nous avons défini une interface ainsi :

```
public interface A {
        public void methode1();
}
```

  - Elle est implémentée dans des dizaines de classes
  - On se rend compte que nous avons besoin d'une nouvelle méthode dans certaines de ces classes (mais pas dans toutes)

- Comment faire ?!

Interfaces qui évoluent (2/2)

- Solution 1

```
public interface A {
      public void methode1();
      public void methode2();
}
```

- Solution 2

```
public interface B extends A {
      public void methode2();
}
```

- Solution 3

```
public interface A {
      public void methode1();
      public default void methode2() {
        //implementation de methode2…
      }
}
```

- Quel impact?

-> il faudra implémenter methode2 dans toutes les classes qui implémentent interface A => il y aura des implémentations vides (*pas bien !*)

- Quel impact?

-> uniquement les classes qui ont besoin de la méthode method2 doivent implémenter l'interface B (*bien !*)

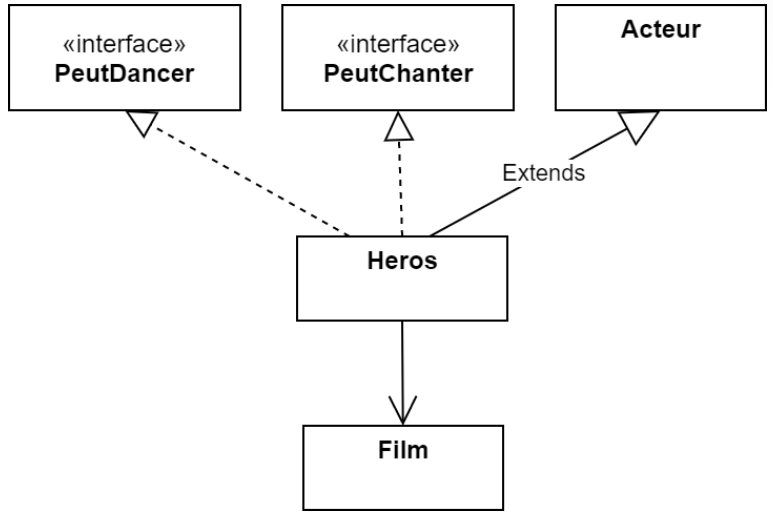
- Quel impact?

-> toutes les classes qui implémentent interface A disposent de la même implémentation de methode2

=> ce n'est pas ce qu'on veut (*pas bien !*)

Interface et héritage multiple (1/2)

```
interface PeutDanser {
   void danse();
}
interface PeutChanter {
   void chante();
}
class Acteur {
   public void parle() { System.out.println("Un Acteur peut parler"); }
}
class Heros extends Acteur implements PeutDanser, PeutChanter {
   public void chante() {System.out.println("Un Heros peut chanter!")}
   public void danse() {System.out.println("Un Heros peut danser!")}
}
```



- Les interfaces permettent l'**héritage multiple**

Une classe peut hériter d'une seule classe mère mais elle peut implémenter plusieurs interfaces

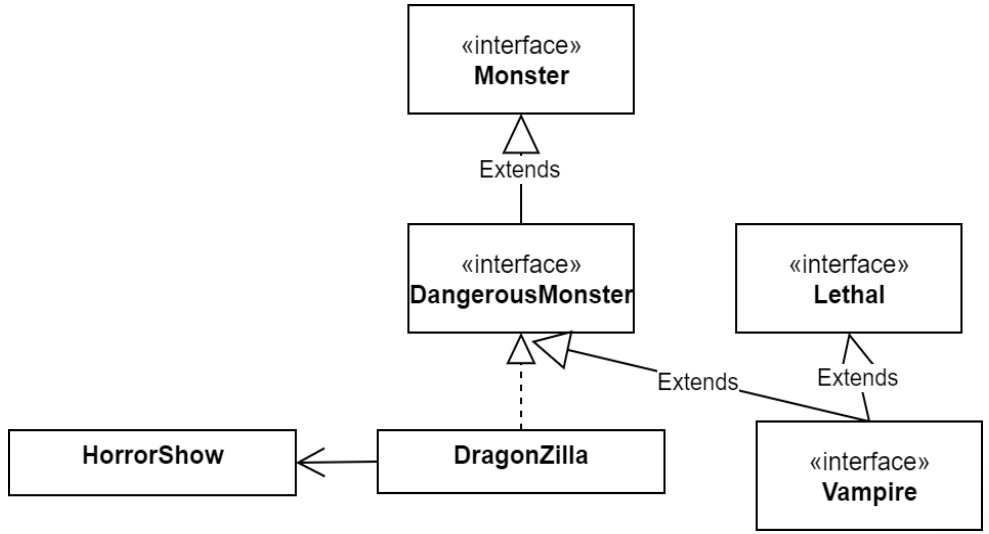
```
public class Film {
   static void danse(PeutDanser x)    { x.danse(); }
   static void chante(PeutChanter x)   { x.chante();  }
   static void parle(Acteur x)         { x.parle();  }
   public static void main(String[] args) {
      Heros h = new Heros();
      danse(h);    //  joue comme un danseur (PeutDanser)
      chante(h);   //  joue comme un chanteur (PeutChanter)
      parle(h);    //  joue comme un Acteur (parle)
   }
}
```

Interface et héritage multiple (2/2)

```
interface Monster {
        void threaten();
}
interface DangerousMonster extends Monster {
        void destroy();
}
interface Lethal {
        void kill();
}
class DragonZilla implements DangerousMonster {
    public void threaten() {
        System.out.println("This monster threatens!");
    }
    public void destroy() {
        System.out.println("This monster destroys!");
    }
}
interface Vampire  extends DangerousMonster, Lethal {
        void drinkBlood();
}
```

```
class HorrorShow {
    static void joueRole (Monster m) {
        m.threaten();
    }
    static void joueRole(DangerousMonster d) {
            d.threaten();
            d.destroy();
    }
    public static void main(String[] args) {
    DragonZilla if2 = new DragonZilla();
    joueRole (if2); //joue comme un Monster
    joueRole (if2); //joue comme un DangerousMonster
  }
}
```

Une interface peut hériter de plusieurs interfaces!



Exercice interface



Cours 3 - Question 6 et Question 7

Plan

1. Rappels
2. Polymorphisme
3. Classes abstraites et Interfaces
4. Exceptions

Les erreurs de programmation

Le programmeur invincible :

« Oui, les erreurs peuvent arriver aux autres,

mais pas dans MON code »



- Et pourtant… ils arrivent car ***errare humanum est***
- Il y a plusieurs types d'erreurs :
  - **Compile-time** (classe non trouvée, perte de précision, erreurs de syntaxe)
  - **Link-time** (librairie dépendante non-trouvée, paramètres non-compatibles entre librairies…)
  - **Run-time** (guillemets manquants, données inattendues, ouverture de fichier inexistant...)
  - **Logique** (opérateurs incorrects)

Les exceptions

- Toute **erreur d'utilisation** potentielle peut provoquer une **erreur de fonctionnement** qui **ne se voit pas à la compilation**
- Une bonne pratique de programmation consiste donc à **vérifier** les entrées fournies par l'utilisateur

```
Scanner sc = new Scanner(System.in);
int number;
System.out.println("Enter a number between 1 and 5 :");
number = sc.nextInt();
While (number < 1 || number > 5) {
    System.out.println("Wrong number !");
    System.out.println("Enter a number between 1 and 5 :");
    number.sc.nextInt();
}
```

Exceptions et méthode (1/2)

- Certaines erreurs se passent dans des méthodes et ne peuvent donc pas être traitées immédiatement
- Solution n°1 : Arrêt du programme

```
public class Homework() {
    private int grade;
    public void setGrade(int grade) {
        if(grade < 0 || grade > 20) {
            System.err.println("Error");   // Message
            System.exit();                 // Quit Program
        } else {
            this.grade = grade;
        }
    }
}
```

- A éviter absolument !!

Exceptions et méthode (2/2)

- Solution n°2 : Renvoyer un statut (OK/KO)

```
public int setGrade(int grade) {
        if(grade < 0 || grade > 20)
                return -1;      // KO
        this.grade = grade;
        return 0;               // OK
}
/*….dans une autre methode: …*/
while (setGrade(number) != 0) {
        System.err.println("Error"); // Message
        number = sc.nextInt();
}
```

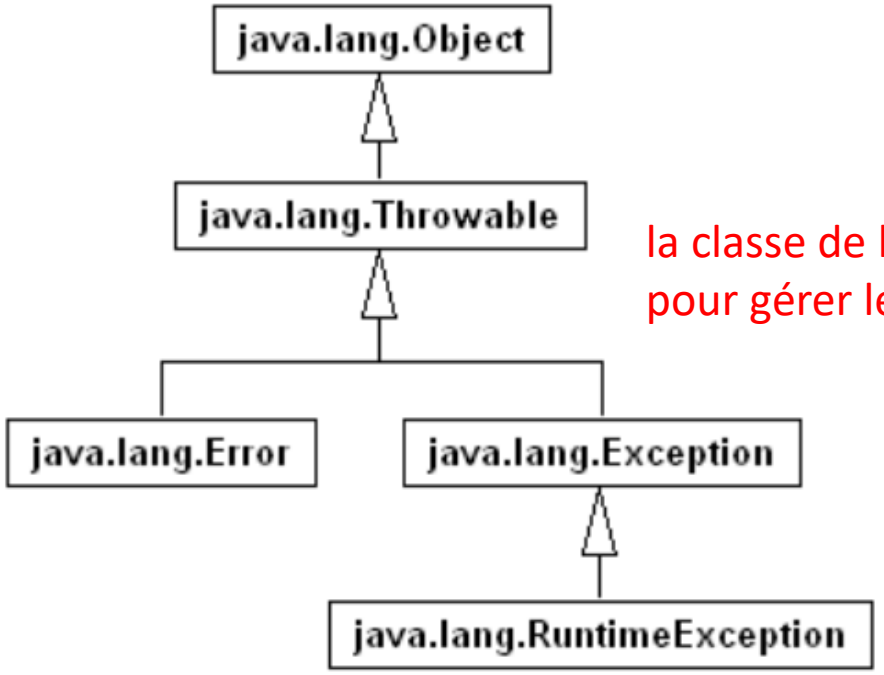
- C’est mieux. Mais nous sommes obligés à utiliser la valeur de retour de la méthode uniquement pour cela

La gestion d'exceptions (1/5)

- En OO, la **gestion d'exceptions**, consiste en un **second canal de communication** entre les méthodes, **dédié à la gestion d'erreur**

- En Java, cette gestion s'appuie sur :
  - la classe `Throwable`
  - le mot clef **`throw`** (envoi d'une exception)
  - le mot clef **`throws`** (qui signale la présence potentielle d'exception)

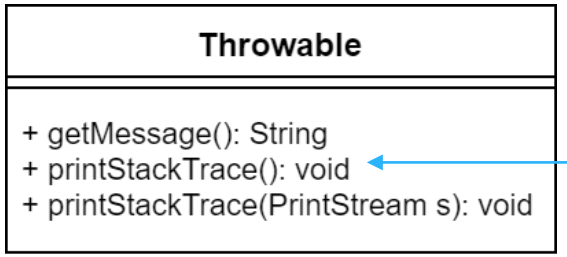
La hiérarchie des exceptions



la classe de base
pour gérer les exceptions

des problèmes liés à
l'environnement
(e.g.OutOfMemoryError)

des erreurs de programmation
(e.g. NullPointerException)
"Unchecked exceptions"



affiche l'exception et l'état de la pile
d'exécution au moment de son appel

On peut créer nos propres exceptions par héritage

La gestion d'exceptions (2/5)

- `throw` est un verbe d'action (à l'impératif) et doit être suivi avec une nouvelle instance d'Exception, qui dérive de `Throwable`
- **C'est cet objet qui sert à transporter le message** et les informations sur la cause de l'erreur

```
public void setGrade(int grade) throws Exception {
	if(grade < 0 || grade > 20)
		throw new Exception("Incorrect grade !");
		//After throw the method ends (similar to return)
	this.grade = grade;   //in case of exception, this line is not executed
}
```

- En Java, `throws` (au présent) est obligatoire **au niveau du prototype** de la méthode pour signaler la présence potentielle d'arrivée d'Exception

La gestion d'exceptions (3/5)

- Maintenant que l'on a vu comment envoyer des `Exception` voyons comment les récupérer . . .

1. Une `Exception` peut-être **capturée** pour être traitée

La capture d'une Exception se fait en utilisant les mots clef

`try` et `catch`

2. Une exception peut-être aussi **renvoyée** directement à la méthode appelante

- **On est forcément dans un des deux cas précédents**

Il est indispensable de toujours capturer (*traiter*) une exception

Que se passe-t-il si on ne la capture pas?

La gestion d'exceptions (4/5)

- Exemple de capture d'exception en Java :

```
public static void main(String[] args) {
    Homework myHomework = new Homework();
    try {
        myHomework.setGrade(10);
        System.out.println("My grade : "+ myHomework.getGrade());
        myHomework.setGrade(24);
        System.out.println("My grade : "+  myHomework.getGrade());
    } catch(Exception e) {
        System.out.println(e.getMessage());
    } finally {
          // le code qu'on met ici s'exécute dans tous les cas
    }
}
```

```
My Homework grade : 10
Incorrect grade !
```

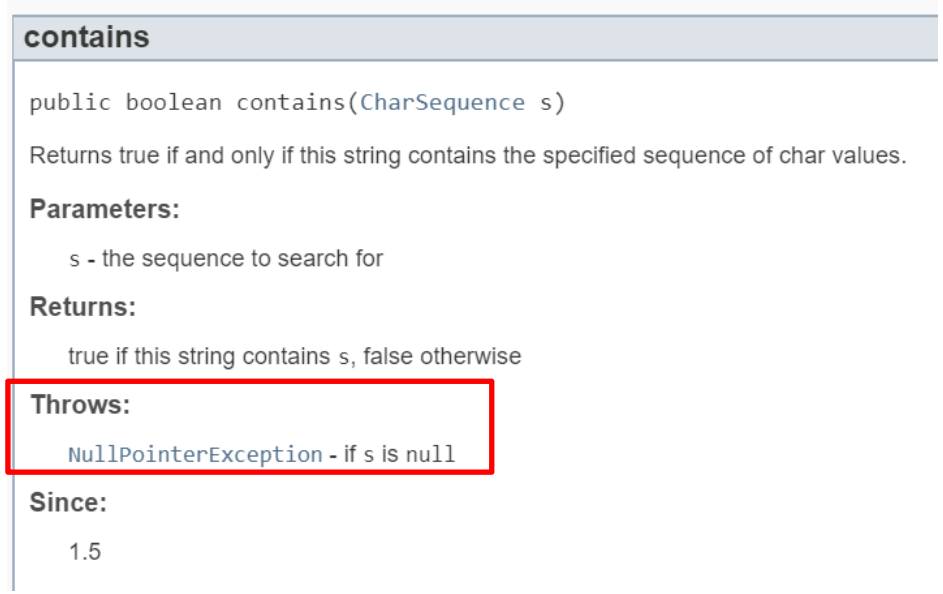
La gestion d'exceptions (5/5)

- Exemple de transmission d'exception en Java :

```
public class Homework {
    public void setGrade(int grade) throws Exception {
        if(grade < 0 || grade > 20)
                throw new Exception("Incorrect grade !");  //méthode exit
        this.grade = grade;
    }
    public void initGrade() throws Exception {
        //l'appellant de initGrade() va devoir s'occuper de l'exception
        this.setGrade(0);
    }
    public static void main(String[] args) {
        Homework myHomework = new Homework();
        try {
                myHomework.initGrade();
        } catch(Exception e) {
                System.out.println(e.getMessage());
        } finally { . . . //le code qu'on met ici s'exécute dans tous les cas }
    }
}
```

Example avec l'API Java

- La méthode `contains` de la classe `java.lang.String`



contains

`public boolean contains(CharSequence s)`

Returns true if and only if this string contains the specified sequence of char values.

Parameters:

`s` - the sequence to search for

Returns:

true if this string contains `s`, false otherwise

Throws:

`NullPointerException` - if `s` is `null`

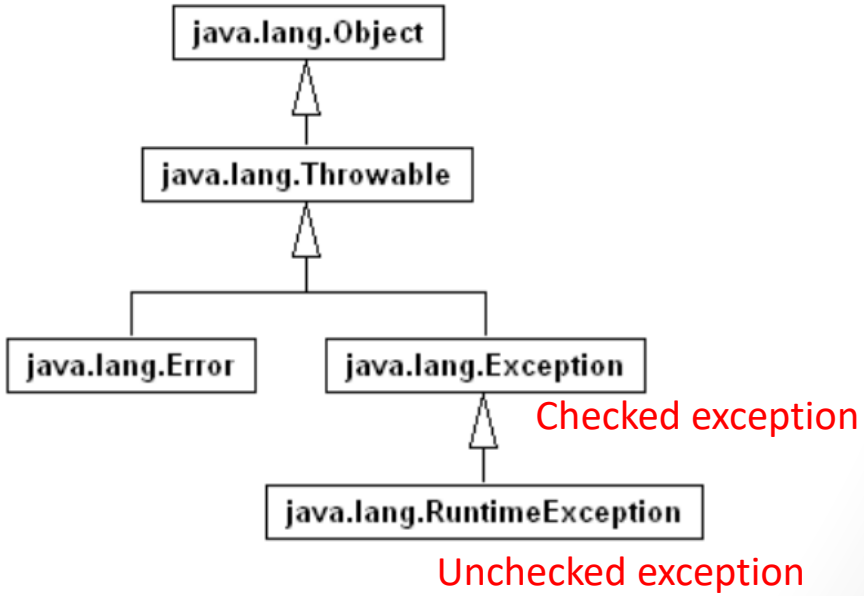
Since:

1.5

```
String phrase = new String("this is my test phrase");
String word = null;          //oubli d'initialisation
try {
      phrase.contains(word);
} catch (NullPointerException ex) { //la catastrophe est évitée!!
      //je gère l'exception ici
}
```

Créer sa classe de gestion des exceptions

- On peut créer nos exceptions en héritant des classes `Exception` ou `RuntimeException`
- On ne peut pas hériter de la classe `Error` (réservée aux erreurs dans la JVM)
- Une « *checked exception* » (`Exception`) est vérifiée à la compilation
- Une « *unchecked exception* » (`RuntimeException`)
  - n'est pas vérifiée à la compilation
  - apparait en général à cause des données erronées
  - Ex. : `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`, `IllegalArgumentException`, `NumberFormatException`



Créer sa classe de gestion des exceptions

```
public class NullPrintException extends RuntimeException {
   // On veut avoir un message personnalisé quand on lève l'exception
   public NullPrintException (String message) {
      // la méthode super invoque les constructeurs de toutes les classes mère
      // ce qui permet de créer la stacktrace complète
      super(message);
   }
}
```

```
public class Printer {
  private Printable currentSheet = null;
  void printSheet (Printable p) {
         p.print();
  }
  void printCurrentSheet() throws NullPrintException {

    if (currentSheet == null)
      throw new
         NullPrintException("La feuille n'est pas chargée!");

    currentSheet.print();
  }
  void setCurrentSheet(Printable p) {
      currentSheet = p;
  }
}
```

```
public class PrinterTest {
    public static void main(String[] args) {
        Printer printer = new Printer();
        Printable p = new Paper();
        printer.setCurrentSheet(p);

        try {
            printer.printCurrentSheet();
        }
        catch (NullPrintException ex) {
            ex.printStackTrace();
        }
    }
}
```

Exercices Exceptions



Cours 3 - Question 8

- Devoir à la maison :
  - Modifiez la `NullPrintException` en *checked exception* et observez ce qui se passe si l'exception n'est pas gérée dans `PrinterTest`


esiea